

Design FB Live Comments

 Real-time Updates

By Evan King · Published Jul 6, 2024 ·  medium · Asked at: 



Try This Problem Yourself

Practice with guided hints and real-time feedback

 Start Practice



Watch Video Walkthrough

Watch the author walk through the problem step-by-step

 Watch Now

Understanding the Problem

What are Facebook Live Comments?

Facebook Live Comments is a feature that enables viewers to post comments on a live video feed. Viewers can see a continuous stream of comments in near-real-time.

Functional Requirements

Core Requirements

1. Viewers can post comments on a Live video feed.
2. Viewers can see new comments being posted while they are watching the live video.
3. Viewers can see comments made before they joined the live feed.

Below the line (out of scope):

- Viewers can reply to comments
- Viewers can react to comments

Non-Functional Requirements

Core Requirements

1. The system should scale to support millions of concurrent videos and thousands of comments per second per live video.
2. The system should prioritize availability over consistency, eventual consistency is fine.
3. The system should have low latency, broadcasting comments to viewers in near-real time (< 200ms end-to-end latency under typical network conditions)

Below the line (out of scope):

- The system should be secure, ensuring that only authorized users can post comments.
- The system should enforce integrity constraints, ensuring that comments are appropriate (ie. not spam, hate

speech, etc.)



🤪 Fun Fact!

Humans generally perceive interactions as real-time if they occur within 200 milliseconds. Any delay shorter than 200 milliseconds in a user interface is typically perceived as instantaneous so when developing real-time systems, this is the target latency to aim for.



🤪 Fun Fact!

The most popular Facebook Live video was called Chewbacca Mom which features a mom thoroughly enjoying a plastic, roaring Chewbacca mask. It has been viewed over 180 million times. Check it out [here](#).

Here's how your requirements section might look on your whiteboard:

Functional Requirements

1. Viewers can post comments
2. Viewers can see all comments posted in near realtime
3. Viewers can see all comments posted before they joined.

----- Below the line -----

- Viewers can reply to comments
- Viewers can react to comments

Non-functional Requirements

1. Scale to millions of viewers per video
2. Availability > consistency
3. Low latency comment broadcast (~200ms)

----- Below the line -----

- Security
- Integrity



FB Live Comments Requirements

The Set Up

Planning the Approach

Before you move on to designing the system, it's important to start by taking a moment to plan your strategy. Fortunately, for these common product style system design questions, the plan should be straightforward: build your design up sequentially, going one by one through your functional requirements. This will help you stay focused and ensure you don't get lost in the weeds as you go. Once you've satisfied the functional requirements, you'll rely on your non-functional requirements to guide you through layering on depth and complexity to your design.

Defining the Core Entities

I like to begin with a broad overview of the primary entities. Initially, establishing these key entities will guide our thought process and lay a solid foundation as we progress towards defining the API. Think of these as the "nouns" of the system.



Why just the entities and not the whole data model at this point? The reality is we're too early in the design and likely can't accurately enumerate all the columns/fields yet. Instead, we start by grasping the core entities and then build up the data model as we progress with the design.

For this particular problem, we only have three core entities:

1. **User:** A user can be a viewer or a broadcaster.
2. **Live Video:** The video that is being broadcasted by a user (this is owned and managed by a different team, but is relevant as we will need to integrate with it).
3. **Comment:** The message posted by a user on a live video.

In your interview, this can be as simple as a bulleted list like:

Core Entities

- Comments
- Live Video
- User



FB Live Comments Core Entities

Now, let's carry on to outline the API, tackling each functional requirement in sequence. This step-by-step approach will help us maintain focus and manage scope effectively.

API or System Interface

We'll need a simple POST endpoint to create a comment.

```
POST /comments/:liveVideoId
Header: JWT | SessionToken
{
  "message": "Cool video!"
}
```



Note that the `userId` is not passed in the request body. Instead, it is a part of the request header, either by way of a session token or a [JWT](#). This is a common pattern in modern web applications. The client is responsible for storing the session token or JWT and passing it in the request header. The server then validates the token and extracts the `userId` from it. This is a more secure approach than passing the `userId` in the request body because it prevents users from tampering with the request and impersonating other users.

We also need to be able to fetch past comments for a given live video.

```
GET /comments/:liveVideoId?cursor={last_comment_id}&pageSize=10&sort=desc
```

Pagination will be important for this endpoint. More on that later when we get deeper into the design.

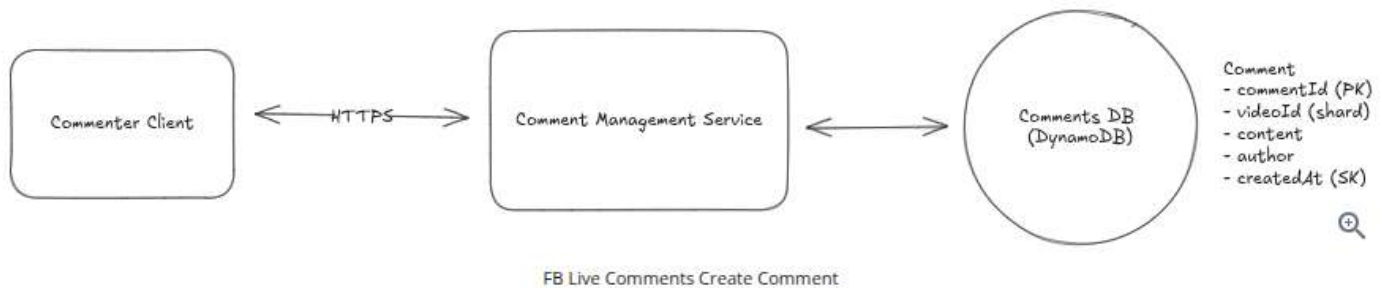
High-Level Design

To get started with our high-level design, let's begin by addressing the first functional requirement.

1) Viewers can post comments on a Live video feed

First things first, we need to make sure that users are able to post a comment.

This should be rather simple. Users will initiate a POST request to the `POST /comments/:liveVideoId` endpoint with the comment message. The server will then validate the request and store the comment in the database.



1. **Commenter Client:** The commenter client is a web or mobile application that allows users to post comments on a live video feed. It is responsible for authenticating the user and sending the comment to the Comment Management Service.
2. **Comment Management Service:** The comment management service is responsible for creating and querying comments. It receives comments from the commenter client and stores them in the comments database. It will also be responsible for retrieving comments from the comments database and sending them to the viewer client -- more on that later.
3. **Comments Database:** For the comments database, we'll choose DynamoDB because it is a fast, scalable, and highly available database. It's a good fit for our use case because we are storing simple comments that don't require complex relationships or transactions, though, other databases like Postgres or MySQL would work here as well.

Let's walk through exactly what happens when a user posts a new comment.

1. The user drafts a comment from their device (commenter client)
2. The commenter client sends the comment to the comment management service via the `POST /comments/:liveVideoId` API endpoint.
3. The comment management service receives the request and stores the comment in the comments database.

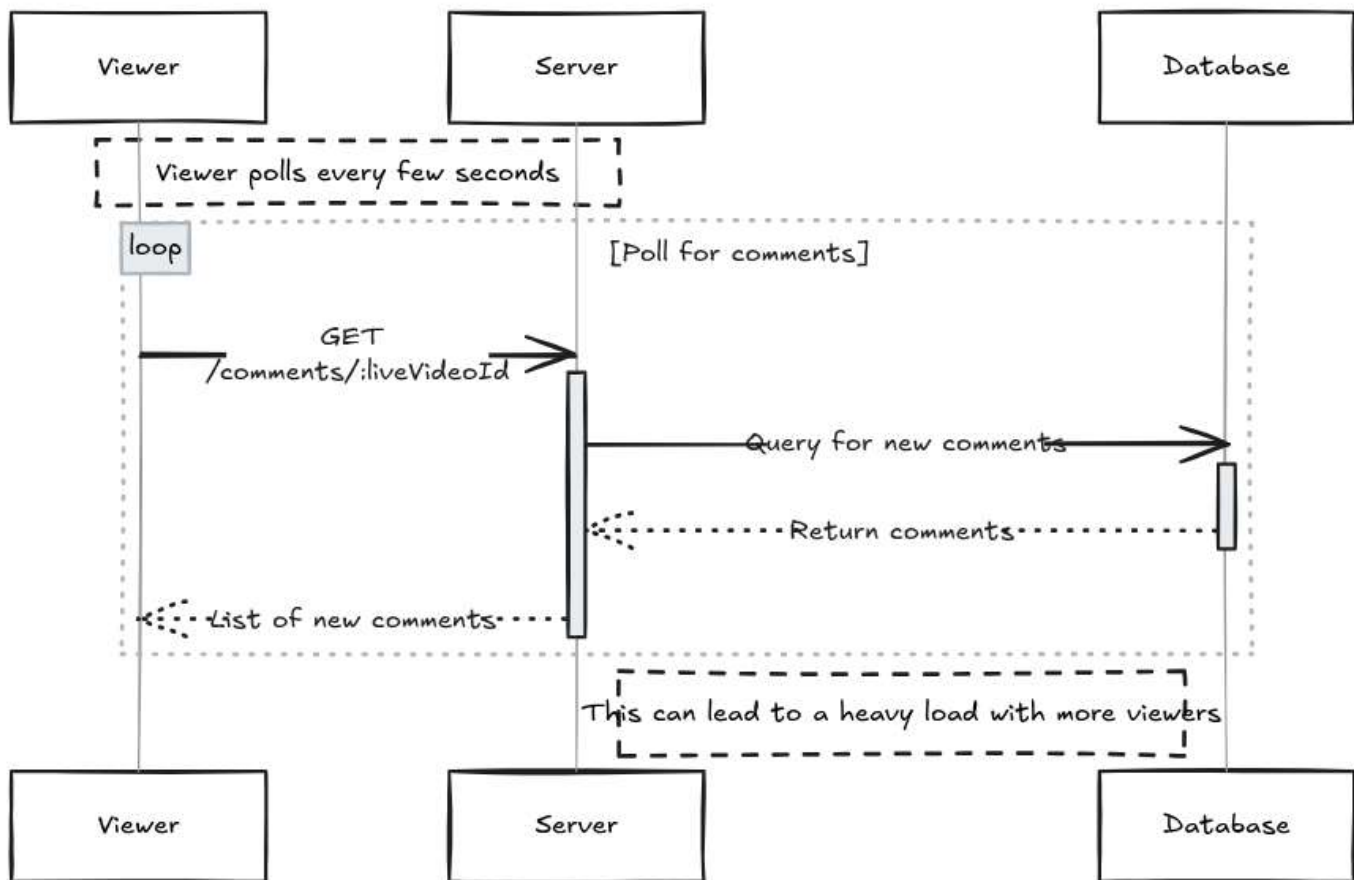
Great, that was easy, but things get a little more complicated when we start to consider how users will view comments.

2) Viewers can see new comments being posted while they are watching the live video.

Now that we've handled comment creation, we need to tackle the challenge of comment distribution - ensuring that when one user posts a comment, all other viewers of the live video can see it.

We can start with the simplest approach: polling.

A working, though naive, approach is to have the clients poll for new comments every few seconds. We would use the `GET /comments/:liveVideoId?since={last_comment_id}` endpoint, adding a `since` parameter to the request that points to the last comment id that the client has seen. The server would then return all comments that were posted after the `since` comment and the client would append them to the list of comments displayed on the screen.



Note: This diagram is illustrative, you would never draw this in a real interview



Polling

This is a start, but it doesn't scale. As the number of comments and viewers grows, the polling frequency will need to increase to keep up with the demand. This will put a lot of strain on the database and will result in many unnecessary requests (since most of the times there will be no new comments to fetch). In order to meet our requirements of "near real-time" comments, we would need to poll the database every few milliseconds, which isn't feasible.



In your interview, if you already know the more accurate, yet complex, solution, you can jump right to it. Just make sure you justify your decision and explain the tradeoffs.

In the case that you are seeing a problem for the first time, starting simple like this is great and sets a foundation for you to build upon in the deep dives.

3) Viewers can see comments made before they joined the live feed

When a user joins a live video, they need two things:

1. They should immediately start seeing new comments as they are posted in real-time
2. They should see a history of comments that were posted before they joined

For the history of comments, users should be able to scroll up to view progressively older comments - this UI pattern is called "infinite scrolling" and is commonly used in chat applications.

We can fetch the initial set of recent comments using our `GET /comments/:liveVideoId` endpoint. While we could use the `since` parameter we added earlier, that would give us comments newer than a timestamp - the opposite of what we want for loading historical comments. What we really want is something that will "give me the N most recent comments that occurred before a certain timestamp".

To do that, we can introduce pagination. Pagination is a common technique used to break up a large set of results into smaller chunks. It is typically used in conjunction with infinite scrolling to allow users to load more results as they scroll down the page.

Whenever you have a requirement that involves loading a large set of results, you should consider pagination.

When it comes to implementing pagination, there are two main approaches: offset pagination and cursor pagination.

Bad Solution: Offset Pagination

Approach

The simplest approach is to use offset pagination. Offset pagination is a technique that uses an offset to specify the starting point for fetching a set of results. Initially, the offset is set to 0 to load the most recent comments, and it increases by the number of comments fetched each time the user scrolls to load more (known as the page size). While this approach is straightforward to implement, it poses significant challenges in the context of a fast-moving, high-volume comment feed.

Example request: `GET /comments/:liveVideoId?offset=0&pageSize=10`

Challenges

First, offset pagination is inefficient as the volume of comments grows. The database must count through all rows preceding the offset for each query, leading to slower response times with larger comment volumes. Most importantly, offset pagination is not stable. If a comment is added or deleted while the user is scrolling, the offset will be incorrect and the user will see duplicate or missing comments.

Great Solution: Cursor Pagination

Approach

A better approach is to use cursor pagination. Cursor pagination is a technique that uses a cursor to specify the starting point for fetching a set of results. The cursor is a unique identifier that points to a specific item in the list of results. Initially, the cursor is set to the most recent comment, and it is updated each time the user scrolls to load more. This approach is more efficient than offset pagination because the database does not need to count through all rows preceding the cursor for each query (assuming we built an index on the cursor field). Additionally, cursor pagination is stable, meaning that if a comment is added or deleted while the user is scrolling, the cursor will still point to the correct item in the list of results.

Example request: `GET /comments/:liveVideoId?cursor={last_comment_id}&pageSize=10`

Since we chose DynamoDB for our comments database, we can use the `last_comment_id` as the cursor via

DynamoDB's `LastEvaluatedKey` feature. The query will look something like this:

```
{
  "TableName": "comments",
  "KeyConditionExpression": "liveVideoId = :liveVideoId AND commentId < :cursor",
  "ExpressionAttributeValues": {
    ":liveVideoId": "liveVideoId",
    ":cursor": "last_comment_id"
  },
  "ScanIndexForward": false,
  "Limit": "pageSize"
}
```

Challenges

While cursor pagination reduces database load compared to offset pagination, it still requires a database query for each new page of results, which can be significant in a high-traffic environment like ours.

Cursor based pagination is a better fit for our use case. Unlike offset pagination, it's more efficient as we don't need to scan through all preceding rows. It's stable - new comments won't disrupt the cursor's position during scrolling. It works well with DynamoDB's `LastEvaluatedKey` feature, and it scales better since performance remains consistent as comment volume grows.

Potential Deep Dives

1) How can we ensure comments are broadcasted to viewers in real-time?

Our simple polling solution was a good start, but it's not going to pass the interview. Instead of having the client "guess" when new comments are ready and requesting them, we can use a push based model. This way the server can push new comments to the client as soon as they are created.

There are two main ways we can implement this. Websockets and Server Sent Events (SSE). Let's weigh the pros and cons of each.

Pattern: Real-time Updates

Facebook Live Comments showcases the **real-time updates pattern** at massive scale. Whether it's broadcasting comments via Server-Sent Events, distributing updates through pub/sub systems, or coordinating across multiple servers, the same principles apply to any system requiring instant data delivery, from collaborative editing to live dashboards to gaming platforms.

[Learn This Pattern](#)

Good Solution: Websockets

Approach

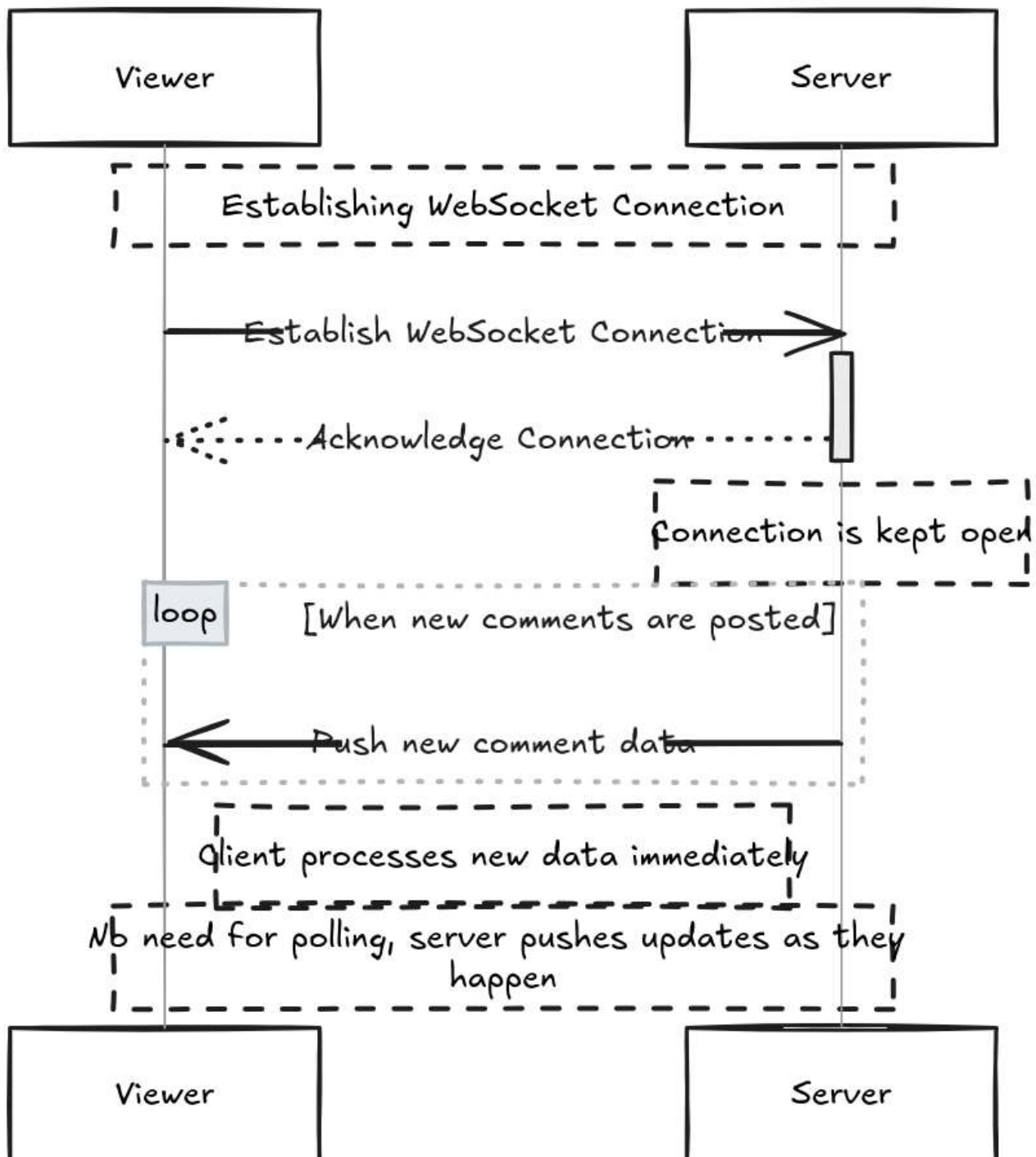
Websockets are a two-way communication channel between a client and a server. The client opens a connection

to the server and keeps it open. The server keeps a connection open and sends new data to the client without requiring additional requests. When a new comment arrives, the Comment Management Server distributes it to all clients, enabling them to update the comment feed. This is much more efficient as it eliminates polling and enables the server to immediately push new comments to the client upon creation.

Challenges

Websockets are a good solution, and for real-time chat applications that have a more balanced read/write ratio, they are optimal. However, for our use case, the read/write ratio is not balanced. Comment creation is a relatively infrequent event, so while most viewers will never post a comment, they will be viewing/reading all comments.

Because of this imbalance, it doesn't make sense to open a two-way communication channel for each viewer, given that the overhead of maintaining the connection is high.



Note: This diagram is illustrative, you would never draw this in a real interview

Websockets

Great Solution: Server Sent Events (SSE)

Approach

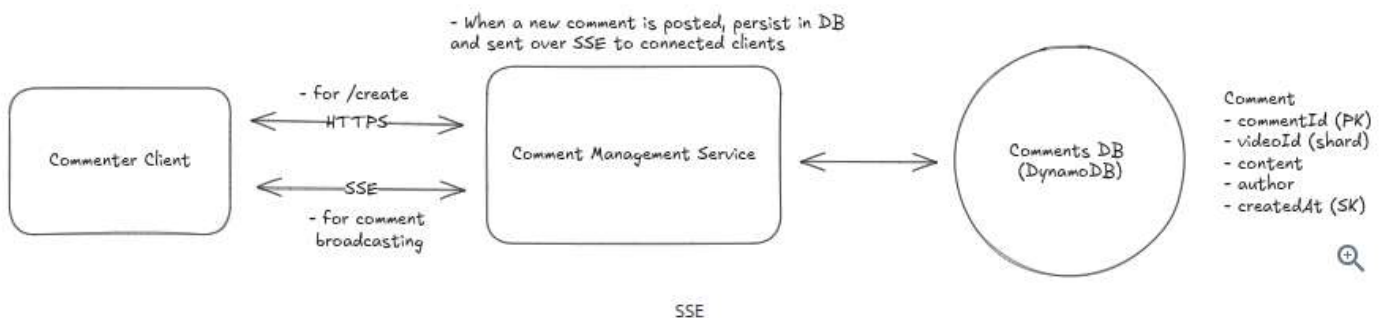
A better approach is to use Server Sent Events (SSE). SSE is a persistent connection that operates over standard HTTP, making it simpler to implement than WebSockets. The server can push data to the client in real-time, while client-to-server communication happens through regular HTTP requests.

This is a better solution given our read/write ratio imbalance. The infrequent comment creation can use standard HTTP POST requests, while the frequent reads benefit from SSE's efficient one-way streaming.

Challenges

SSE introduces several infrastructure challenges that need careful consideration. Some proxies and load balancers lack support for streaming responses, leading to buffering issues that can be difficult to track down. Browsers also impose limits on concurrent SSE connections per domain, which creates problems for users trying to watch multiple live videos simultaneously. The long-lived nature of SSE connections complicates monitoring and debugging efforts.

SSE connections may be terminated by proxy servers and load balancers after periods of inactivity, but modern browsers handle this gracefully through automatic reconnection and the Last-Event-ID header, ensuring no comments are missed during reconnection.



Here is our updated flow:

1. User posts a comment and it is persisted to the database (as explained above)
2. In order for all viewers to see the comment, the Comment Management Service will send the comment over SSE to all connected clients that are subscribed to that live video.
3. The Commenter Client will receive the comment and add it to the comment feed for the viewer to see.

2) How will the system scale to support millions of concurrent viewers?

We landed on Server Sent Events (SSE) being the appropriate technology. Now we need to figure out how to scale it. With SSE, we need to maintain an open connection for each viewer. Modern servers and operating systems can handle large numbers of concurrent connections—commonly in the range of 100k. Realistically, system resources like CPU, memory, and file descriptors become the bottleneck before you hit any theoretical limit. If we want to support many millions of concurrent viewers, we simply won't be able to do it on a single machine. We must scale horizontally by adding more servers.

The question then becomes how do we distribute the load across multiple servers and ensure each server knows which comments to send to which viewers?



Contrary to a common misconception, the capacity isn't limited to 65,535 connections. That number refers to the range of port numbers, not the number of connections a single server port can handle. Each TCP connection is identified by a unique combination of source IP, source port, destination IP, and destination port. With proper OS tuning and resource allocation, a single listening port can handle hundreds of thousands or even millions of concurrent SSE connections.

In practice, however, the server's hardware and OS limits—rather than the theoretical port limit—determine the maximum number of simultaneous connections.

Before we dive into solutions, let's understand the core challenge with horizontal scaling:

When we add more servers to handle the load, viewers watching the same live video may end up connected to different servers. For example:

- UserA is watching Live Video 1 and connected to Server 1
- UserB is watching Live Video 1 but connected to Server 2

Now imagine a new comment is posted on Live Video 1. If this comment request hits Server 1:

- Server 1 can easily send it to UserA since they're directly connected
- But Server 1 has no way to send it to UserB, who is connected to Server 2

This is our key challenge: How do we ensure all viewers see new comments, regardless of which server they're connected to?

Bad Solution: Horizontal Scaling with Load Balancer and Pub/Sub



Approach

The first thing we need to do is separate out the write and read traffic by creating Realtime Messaging Servers that are responsible for sending comments to viewers. We separate this out because the write traffic is much lower than the read traffic and we need to be able to scale the read traffic independently.

To distribute incoming traffic evenly across our multiple servers we can use a simple load balancing algorithm like round robin. Upon connecting to a Realtime Messaging Server through the load balancer, the client needs to

send a message informing the server of which live video it is watching. The Realtime Messaging Server then updates a mapping in local memory with this information. This map would look something like this:

```
{
  "liveVideoId1": ["sseConnection1", "sseConnection2", "sseConnection3"],
  "liveVideoId2": ["sseConnection4", "sseConnection5", "sseConnection6"],
  "liveVideoId3": ["sseConnection7", "sseConnection8", "sseConnection9"],
}
```

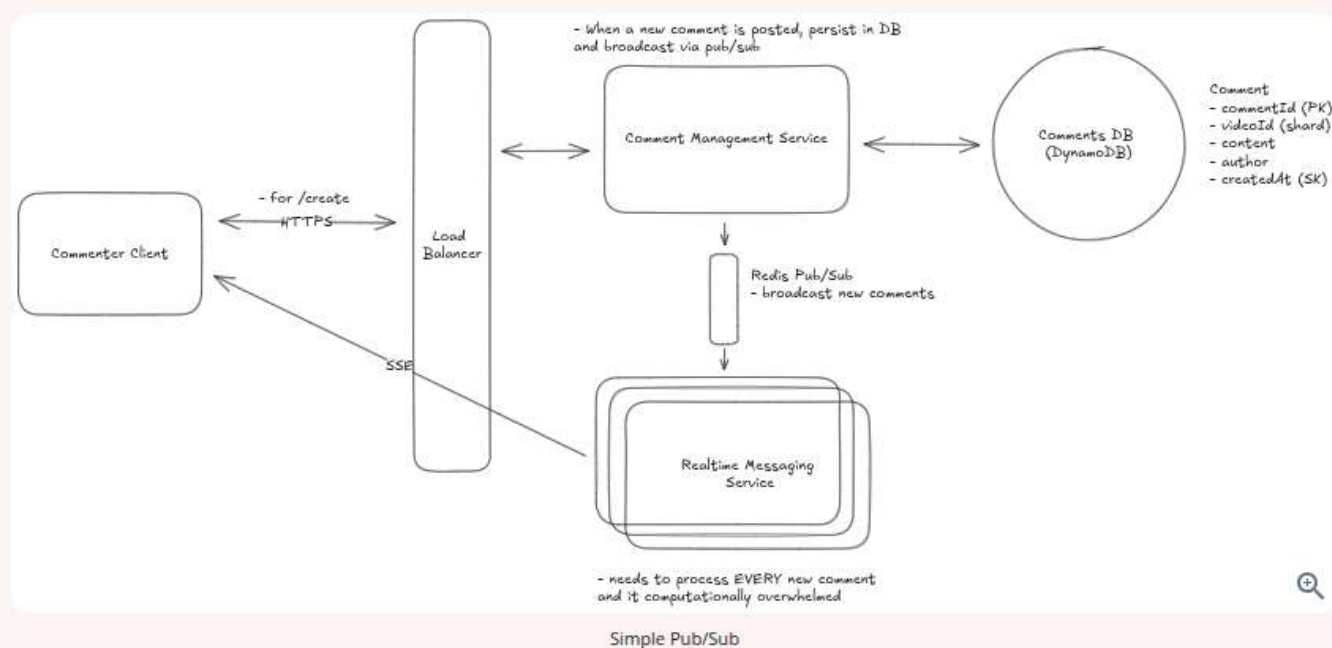
Where `sseConnection{N}` is a pointer to the SSE connection for that viewer. Now, anytime a new comment is created, the server can loop through the list of viewers for that live video and send the comment to each one.

The question then becomes how does each Realtime Messaging Server know that a new comment was created? The most common solution is to introduce pub/sub.

A pub/sub system is a messaging system that uses a publish/subscribe model. Publishers send messages to a channel or topic, and subscribers receive messages from a channel or topic. In our case, the comment management service would publish a message to a channel whenever a new comment is created. The Realtime Messaging Servers would all subscribe to this channel and receive the message. They would then send the comment to all the viewers that are watching the live video.

Challenges

This approach works, but it is not very efficient. Each Realtime Messaging Server needs to process every comment, even if it is not broadcasting the live video. This leads to inefficiency, slow performance, and high compute intensity that's impractical at FB scale.



Good Solution: Pub/Sub Partitioning into Channels per Live Video

Approach

To improve upon the previous approach, we can partition the comment stream into different channels based on the live video. Each Realtime Messaging Server would subscribe only to the channels it needs, determined by the viewers connected to it.

Since having a channel per live video will consume a lot of resources and, in some cases like with Kafka may even be infeasible, we can use a hashing function to distribute the load across multiple channels.

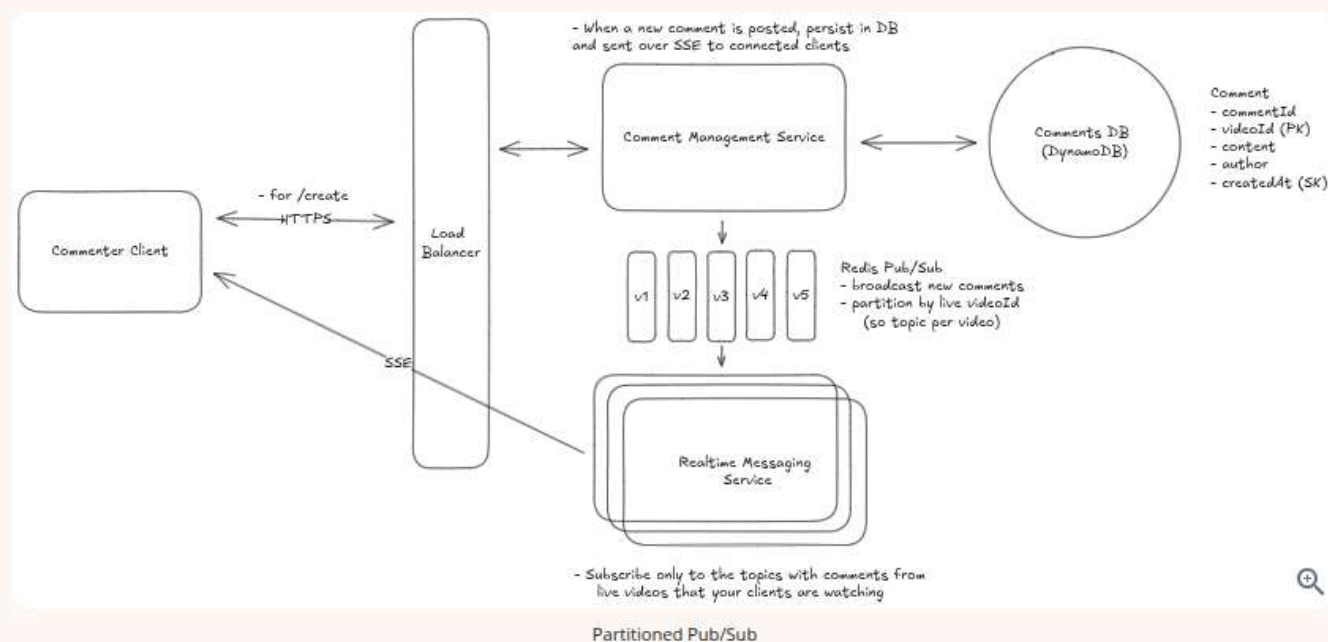
We can create N channels and determine which channel to broadcast a comment to based on

```
hash(liveVideoId) % N.
```

This ensures there is a reasonable upper bound on the number of channels.

Challenges

While this approach is more efficient, it is not perfect. With the load balancer using round robin, there's a risk that a server could end up with viewers subscribed to many different streams, replicating the issue from the previous approach.



Great Solution: Partitioned Pub/Sub with Layer 7 Load Balancer

Approach

To address the issue of servers handling viewers from many different live videos, we need a more intelligent allocation strategy. The goal is to have each server primarily handle viewers of the same live video, making it easier to limit the number of topics or channels each server needs to subscribe to.

There are two ways we can achieve this allocation, both require some scripting or configuration.

Option 1: Intelligent Routing via Scripts or Configuration

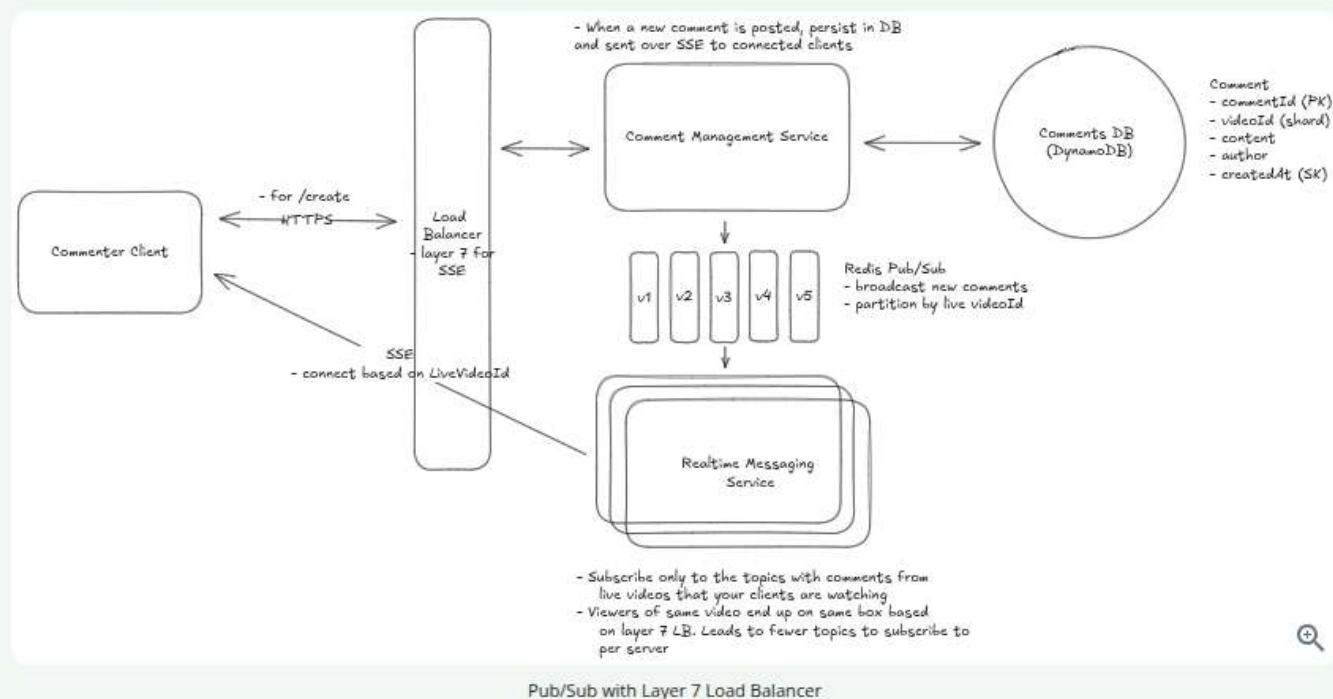
A Layer 7 load balancer or API gateway can leverage **consistent hashing** based on the `liveVideoId`. By inspecting the request—such as a header or path parameter that includes the `liveVideoId`—the load balancer

can apply a hashing function that always routes viewers of the same video to the same server. Tools like NGINX or Envoy can be scripted with Lua or configuration directives to achieve this content-based routing, reducing overhead and keeping related viewers together.

Option 2: Dynamic Lookup via a Coordination Service (e.g., ZooKeeper)

Instead of relying on hashing alone, we can store a dynamic mapping of `liveVideoId` to a specific server in a coordination service like Zookeeper. The load balancer or custom proxy layer queries Zookeeper to determine the correct server for each `liveVideoId`. As traffic patterns change, new servers can register themselves or existing mappings can be updated in Zookeeper, and the load balancer will route future requests accordingly. This approach offers more flexibility but introduces operational complexity and requires caching and careful failure handling.

The reality is this is pretty complex, and you would not be expected to go into too much detail here in an interview. Just explaining that you need a way to colocate viewers of the same live video to the same server is good enough. Then mention what some high level options would be.



Advanced candidates may point out the tradeoffs in different pub/sub systems. For example, Kafka is a popular pub/sub system that is highly scalable and fault-tolerant, but it has a hard time adapting to scenarios where dynamic subscription and unsubscription based on user interactions, such as scrolling through a live feed or switching live videos, is required. Redis pub/sub provides low latency but offers no message persistence (fire-and-forget), which could lead to missed messages during disconnections. Kafka, while having higher latency, provides message persistence and exactly-once delivery semantics, making it more suitable for scenarios where message delivery guarantees are critical.

However, the main concern with Redis is its potential for data loss due to periodic disk writes and the challenges of memory limitation, which could be a bottleneck for scalability. Additionally, while Redis offers high availability configurations like Redis Sentinel or Redis Active-Active, these add to the operational complexity of managing a Redis-based system. The pub/sub solution is the "correct academic answer" and should clearly pass the interview, but the reality is choosing the right pub/sub system is a complex decision that requires a deep understanding of the system's requirements and tradeoffs.

Great Solution: Scalable Dispatcher Instead of Pub/Sub



Approach

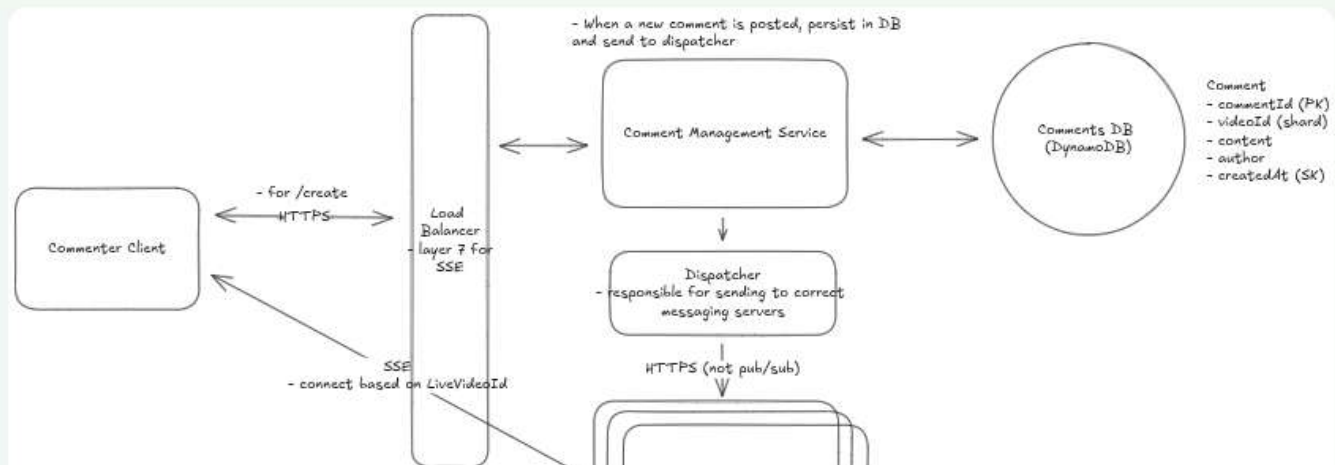
All our previous approaches relied on a pub/sub system, where each Realtime Messaging Server subscribes to topics and receives comments as they're published. This means the comment creation service doesn't need to know the destination server; it just publishes messages and the right servers pick them up. While standard and effective, this approach adds complexity in managing subscriptions and scaling read-heavy workloads.

An alternative approach inverts the model: the service creating comments directly routes each new comment to the correct Realtime Messaging Server. To achieve this, we introduce a Dispatcher Service. Instead of broadcasting to topics, the Dispatcher Service determines the exact server responsible for the viewers of a given live video and sends the comment there.

So when a new comment comes in, the receiving server asks the Dispatcher Service, "What are all the other servers I should send this comment to?"

At a high-level, here is how it works:

- 1. Maintaining Dynamic Mappings:** The Dispatcher Service keeps a dynamic map of which Realtime Messaging Server is responsible for which liveVideoId. This can be stored in memory and periodically refreshed from a consistent store (like Zookeeper or etcd), or updated via heartbeats and registration protocols as servers join or leave.
- 2. Registration & Discovery:** When a Realtime Messaging Server comes online, it registers with the Dispatcher Service (or the coordination store it consults). The Dispatcher updates its internal mapping to reflect which server is now serving particular live videos. Likewise, if a server scales down or fails, the Dispatcher updates the mapping accordingly.
- 3. Direct Routing:** Upon receiving a new comment, the comment management service calls the Dispatcher Service to determine the correct server. The Dispatcher uses its current mapping and forwards the comment directly to the responsible Realtime Messaging Server, which then sends it to connected viewers.
- 4. Scalability & Redundancy:** In high-demand scenarios, multiple Dispatcher instances can run in parallel behind a load balancer. They all consult the same coordination data, ensuring consistency. This replication adds redundancy, making the system more resilient to failures.





Dispatcher Service

Challenges

The main challenge is ensuring the Dispatcher Service's mapping is accurate and up-to-date. Rapid changes in viewer distribution (e.g., a viral stream causing a sudden influx of viewers) require that the Dispatcher's view of the system be refreshed frequently. This might involve coordination services like Zookeeper or etcd, which must efficiently propagate changes to all Dispatcher instances.

Another difficulty is maintaining consistency across multiple Dispatcher instances. If they each cache routing information, updates must be coordinated to prevent stale mappings. Strong consistency can be achieved through a distributed coordination service, careful cache invalidation policies, and possibly a leader election mechanism to manage updates safely.

Both the pub/sub approach with viewer co-location and the dispatcher service approach are great solutions. The pub/sub is typically easier with fewer corner cases, so it is the one I would use in an interview, but both are "correct" answers.

What is Expected at Each Level?

Ok, that was a lot. You may be thinking, "how much of that is actually required from me in an interview?" Let's break it down.

Mid-level

Breadth vs. Depth: A mid-level candidate will be mostly focused on breadth (80% vs 20%). You should be able to craft a high-level design that meets the functional requirements you've defined, but many of the components will be abstractions with which you only have surface-level familiarity.

Probing the Basics: Your interviewer will spend some time probing the basics to confirm that you know what each component in your system does. For example, if you add an API Gateway, expect that they may ask you what it does and how it works (at a high level). In short, the interviewer is not taking anything for granted with respect to your knowledge.

Mixture of Driving and Taking the Backseat: You should drive the early stages of the interview in particular, but the interviewer doesn't expect that you are able to proactively recognize problems in your design with high precision. Because of this, it's reasonable that they will take over and drive the later stages of the interview while probing your design.

The Bar for FB Live Comments: For this question, I expect that candidates proactively realize the limitations with a polling approach and start to reason around a push based model. With only minor hints they should be able to come up with the pub/sub solution and should be able to scale it with some help from the interviewer.

Senior

Depth of Expertise: As a senior candidate, expectations shift towards more in-depth knowledge — about 60% breadth and 40% depth. This means you should be able to go into technical details in areas where you have hands-on experience. It's crucial that you demonstrate a deep understanding of key concepts and technologies relevant to the task at hand.

Advanced System Design: You should be familiar with advanced system design principles. For example, knowing how to use pub/sub for broadcasting messages. You're also expected to understand some of the challenges that come with it and discuss detailed scaling strategies (it's ok if this took some probing/hints from the interviewer). Your ability to navigate these advanced topics with confidence and clarity is key.

Articulating Architectural Decisions: You should be able to clearly articulate the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability. You justify your decisions and explain the trade-offs involved in your design choices.

Problem-Solving and Proactivity: You should demonstrate strong problem-solving skills and a proactive approach. This includes anticipating potential challenges in your designs and suggesting improvements. You need to be adept at identifying and addressing bottlenecks, optimizing performance, and ensuring system reliability.

The Bar for Fb Live Comments: For this question, E5 candidates are expected to speed through the initial high level design so you can spend time discussing, in detail, how to scale the system. You should be able to reason through the limitations of the initial design and come up with a pub/sub solution with minimal hints. You should proactively lead the scaling discussion and be able to reason through the trade-offs of different solutions.

Staff+

Emphasis on Depth: As a staff+ candidate, the expectation is a deep dive into the nuances of system design — I'm looking for about 40% breadth and 60% depth in your understanding. This level is all about demonstrating that, while you may not have solved this particular problem before, you have solved enough problems in the real world to be able to confidently design a solution backed by your experience.

You should know which technologies to use, not just in theory but in practice, and be able to draw from your past experiences to explain how they'd be applied to solve specific problems effectively. The interviewer knows you know the small stuff (REST API, data normalization, etc) so you can breeze through that at a high level so you have time to get into what is interesting.

High Degree of Proactivity: At this level, an exceptional degree of proactivity is expected. You should be able to identify and solve issues independently, demonstrating a strong ability to recognize and address the core challenges in system design. This involves not just responding to problems as they arise but anticipating them and implementing preemptive solutions. Your interviewer should intervene only to focus, not to steer.

Practical Application of Technology: You should be well-versed in the practical application of various technologies. Your experience should guide the conversation, showing a clear understanding of how different tools and systems can be configured in real-world scenarios to meet specific requirements.

Complex Problem-Solving and Decision-Making: Your problem-solving skills should be top-notch. This means not only being able to tackle complex technical challenges but also making informed decisions that consider various factors such as scalability, performance, reliability, and maintenance.

Advanced System Design and Scalability: Your approach to system design should be advanced, focusing on

scalability and reliability, especially under high load conditions. This includes a thorough understanding of distributed systems, load balancing, caching strategies, and other advanced concepts necessary for building robust, scalable systems.

The Bar for FB Live Comments: For a staff+ candidate, expectations are high regarding depth and quality of solutions, particularly when it comes to scaling the broadcasting of comments. I expect staff+ candidates to not only identify the pub/sub solution but proactively call out the limitations around reliability or scalability and suggest solutions. They likely have a good understanding of the exact technology they would use and can discuss the trade-offs of different solutions in detail.

Mark as read ☐

[Next: Design FB Post Search](#) →

How would you rate the quality of this article?

1

2

3

4

5

6

7

8

9

10

PoorGreat

Add a comment...

B *I* `</>`

Comment

☐ Anonymous

Posting as leetcode

🔍 Search 370 comments

Sort By
Popular ▼

U **UniversalCyanQuelea456** ★ Top 1% • 1 year ago

One observation: the real time/low latency requirement matters less and less the more users there are in a chat. For streams with millions of views it barely matters at all.

I actually think the behavior for high-volume chats is very different to low-volume ones (which basically looks identical to Design WhatsApp/Discord/Telegram).

If the average comment is 4 words and there are 5000 comments/sec, reading them would be equivalent to reading War and Peace in 29 seconds. If we show the last 20 messages on screen, the average message will be on screen for 4 milliseconds.

There's no conversation going on, so we don't care about causality, total order broadcast or reliability of delivering individual messages really, at least for the "live" tail of comments.

And lastly, if we think about what's going to happen in the client. Are we going to ask them to stream 4Mbps of messages to the device, only to immediately trash it? We're probably only going to keep a few hundred messages in memory, which may be only 0.1 seconds worth of the data stream.

So my thought for these huge streams is: At some threshold flip from SSE to a CDN. Use a ring buffer with a capacity of 100-200 messages and take a snapshot every second and put it in Redis. Clients stream the messages from the CDN, which hits our API and in the client we can simulate a constant stream instead of jerky UI updates by spreading out the message UI updates. This guarantees users have a smooth experience and stable business KPIs.

Show More

👍 110 [Reply](#)

N **nicolasi** ★ Top 5% • 1 year ago

studying for a Meta Interview loop, and reading all these comments is like reading war and peace once per day, for a whole month

👍 10 [Reply](#)



Evan King Admin • 11 months ago

lol

👍 0 [Reply](#)



Evan King Admin • 1 year ago

This rocks! Completely agree. I'll add a todo to myself to come back and add this to the deep dive.

👍 7 [Reply](#)



Anmol Rattan • 1 year ago

Just following up, if we updated the deep-dive on this?

👍 3 [Reply](#)



Evan King Admin • 1 year ago

Not yet :)

👍 1 [Reply](#)



CuddlyBlueFalcon150 Premium • 1 month ago

Evan - following up on this popular deep dive. We will take it as this year's Christmas present from you ;)

👍 6 [Reply](#)



satishm752 Premium • 3 months ago

Can you please add deep dive on this?

👍 5 [Reply](#)



Hikaru Premium • 1 year ago

When the number of comments is too high, can we use a rate limiter for comments (say 20 comments per second) so that it can be handled by the SSE approach? Then we don't need to switch between SSE and polling to simplify our design?

👍 0 [Reply](#)



GradualSalmonBooby605 • 1 year ago

A rate limiter per user is essential however, setting a limit of 20 comments per second per video will create a poor user experience. In a popular live stream with millions of viewers, each user might only be able to send 1 comment out of 20, 30, or even 100 attempts. For the other attempts, we would need to display a message saying "try again."

Regarding the UniversalCyanQuelea456 comment, that's a great observation. We could also implement a "Read Your Own Write" to enhance user experience. For example, when I send a comment, I can see it immediately while others cant. We can also use the client for this purpose without introducing additional complexity on the server side. Since the number of comments is high, refreshing the page essentially turns my comment into history.

One more point: what if we don't show comments to all users? It doesn't really matter if all users see my comment! even if we share it with a random group, that should still be acceptable unless a single state for all users at any given time is part of the requirements. It provides us with significant flexibility in broadcasting the message.

👍 1 [Reply](#)

D

DevelopedBlushTyrannosaurus417 • 1 year ago

This is amazing! I am learning so much from the comments!
 Evan/Stefan : Please add an upvote feature for comments.

 2 [Reply](#)
**Evan King** Admin • 1 year ago

Good idea! Added to backlog :)

 4 [Reply](#)

S

socialguy ★ Top 5% • 1 year ago

Clients stream the messages from the CDN, which hits our API

I don't understand, the whole point of CDN is to avoid hitting the origin server.

 1 [Reply](#)

C

CulturalRoseSturgeon964 • 1 year ago

It doesn't hit it on every call. The CDN caches the response.

 0 [Reply](#)

N

Nitesh Tiwari • 1 year ago

What a cool insight, thanks for sharing

 0 [Reply](#)

C

core2extremist ★ Top 10% • 1 year ago

THANKS!

When I read the NFRs on this problem I was thinking it meant up to 10M viewers in one stream with 10k comments per second.

At that scale I think an even more serious issue is we need dynamic fanout. At 20B/comment (very efficient protocol, compression) we'd have 200KB per user and thus a modest cloud instance with 20 MB/s of bandwidth would only sustain 100 users. Thus we would need 100k cloud instances. Then the problem is if we have one master broadcasting messages then we'd need $\log(10M)/\log(100) = 3-4$ levels of fanout.

THEN the challenge (are we tired of challenges yet?? lol) is when we start the stream we don't want to have all those servers yet, we don't want to have order 100k instances running unless actually necessary. So as we add instances to adapt to traffic we'd need dynamic fanout.

At the one-hour mark in the practice interview I was lost in the weeds of the deep dive, trying to answer the obvious question "so how is dynamic fanout going to work?" Somewhat similar to what Evan suggested for load balancing. We'd have a pool of instances ready to accept connections. We'd have a dynamic fanout service that has a master server for each video, that's where the writes ultimately go. Each server either have fanout connections or direct connections. When we add fanouts and direct connections, add them to the lowest level of the tree possible to minimize latency. When we add

[Show More](#)
 0 [Reply](#)

D

Derek Brown • 1 month ago

I had very similar thoughts with respect to large streams. My idea was that as the size of a stream scales, I would first start batching comments together. If it gets large enough, I would split the comments essentially into separate chatrooms (add a random salt to the stream ID and partition on this). Users would consistently see comments from the same set of people, but would not see all of the comments on the video.

👍 0 [Reply](#)

L **LogicalAmethystToad166** ★ Top 10% • 11 months ago

Hey Evan!

Thanks for such good content! We really appreciate it. I think there is a slight mistake in the DynamoDB schema... You have

Comments

commentId (PK)

videoId (shard)

content

author

createdAt (sort key)

I understand that we want to shard it or partition it based on videoId so that all comments of a video go to the same partition and we avoid any scatter gather anti-pattern etc. However, you put "commentId" as the PK (partition key), and that's what's gonna be used as sharding, so different comments from the same video will go to different partitions... I think you meant to make "VideoId" the Partition Key (PK) here...

so, from DynamoDB technical standpoint it should have been like the following:

Comments

[Show More](#)

👍 14 [Reply](#)

S **stevenjern** • 8 months ago

I don't think you can use videoId in this table as primary key because it's not unique.

👍 0 [Reply](#)

Q **CoffeeLover** Premium • 2 months ago

VideoId is the partition key, and along with commentId as sort key to guarantee uniqueness. Partition key alone doesn't have to be unique

👍 1 [Reply](#)

M **Mohammad Asif** ★ Top 10% • 1 year ago

All components (streams, db, RMS) are partitioned based on VideoId. How are we handling a uneven distribution of load (hot key) issue here i.e. one video having millions of live comments coming in concurrently while others have hundreds?

Switching to userId based partitioning wouldn't be great either because it would lead to scatter gather pattern queries across all partitions to fetch all comments for a video.

Another approach could be adding a random suffix to videoIds for popular videos (lets between 1 and 5) and that way we increase the capacity for a popular video by 5x. This will help with hot key issue at the cost of more complexity.

Are there other solutions for handling hot key issue in this problem.

👍 10 [Reply](#)

M **Mohammad Asif** ★ Top 10% • 1 year ago

Another soln could be like adding a queue to buffer incoming comments before they are published to storage. Downside is this would increase the delay at which the comments will be available to readers.

This will not solve the traffic skewness problem. but prevent it from browning out our systems.

👍 3 [Reply](#)

Evan King Admin • 1 year ago

You nailed it! You listed off the main considerations I would raise in an interview. I'd avoid the queue given the realtime requirement and instead opt for further sharding.

👍 1 [Reply](#)

OlympicBlackLynx843 ★ Top 1% • 11 months ago

Seems like we might be overlooking the scale and throughput involved - I didn't see any back of the envelope math.

Quoting from NFR: "The system should scale to support millions of concurrent videos, and thousands of comments, per second, per live video".

That's 1B writes per second (Incoming new comments). How does the http/RESTful 'comment CRUD' service handle the writes to DB at this rate? Assuming our service is stateless, even with 1000+ nodes, with a round-robin LB in front of it, each node will receive 1M new comment writes per second.

We also need 1000 nodes in a sharded DynamoDB deployment - even with that, can a single DynamoDB node handle 1M writes/sec? Likely not in my estimation - 100k writes/second might be a stretch.

To make matter worse, the reads are 1000x the writes (assuming 1000 viewers for each live video on average, peaks could be more like 1M viewers for one live video!). That's 1 Trillion updates over SSE per second. How big does the pub/sub server farm and SSE server farm need to be, even if its sharded (by videoid as suggested), to handle 1 Trillion reads/second total?

Am I doing something wrong in the math above?

[Show More](#)

👍 8 [Reply](#)

OlympicBlackLynx843 ★ Top 1% • 11 months ago

I did some more research so wanted to add that something like Zoom for e.g is known to have (by some estimates) 150k servers globally! So proposing that we might need 10k or even 100k servers for massive scales like in this case ('youtube - live comments') is probably quite reasonable. Google can afford it.

And a startup that is aspiring for that kind of scale can scale up slowly.

That said, I am sure they also use other creative techniques to throttle the reads (using CDN with polling as some have suggested here?) as well as the writes (using queues or data streaming platforms?) in the real-world - where you are not designing systems in theory; you are actually expected to design, build, run and operationalize! Any comments or perspective on this from Stefan, Evan and team would be greatly appreciated.

👍 1 [Reply](#)

ZealousScarletCobra427 • 1 year ago

Right - but increasing the shards at runtime would require manual intervention, since we won't know beforehand which videos can become popular.

👍 0 [Reply](#)

SurvivingCrimsonMarten169 • 1 year ago

Appending number if fine but make sure you think about the read path. Let's use ddb for comment table as example. Each partition has a hard limit on 1K WCU. If we want to add a random number and support 50k/sec write, we will add 0-50. The read part needs to query 50 partitions and merge the result. It could be time consuming.

👍 1 [Reply](#)

Mohammad Asif ★ Top 10% • 1 year ago

We could also batch comments to handle high volume comments for popular live videos

👍 0 [Reply](#)

R **Ryan Zhang** • 9 months ago

So if we added a random suffix for popular videos, and a comment came in from a client connected to partition 3, how would we know how to fan that comment out the clients connected to partitions 1 through 5?

👍 0 [Reply](#)

N **Rick Sanchez** ★ **Top 10%** • 6 months ago

This talk by a LinkedIn engineer explains dispatcher mechanism for live video reactions very well (Streaming a Million Likes/Second: Real-Time Interactions on Live Video) <https://www.youtube.com/watch?v=yqc3PPmHvRA>

👍 9 [Reply](#)

L **LabourlvoryPython699** **Premium** • 1 month ago

thanks for sharing!

👍 0 [Reply](#)

Justin Schulz • 11 months ago

Sorry to be that guy in the comments, but... Even though I agree that RESTful semantics are not the most important thing, and you've accounted for pluralization, wouldn't it be more like the following?

POST /liveVideos/:liveVideoId/comments

Great video as always, though, just a minor nitpick I wanted to clarify since you brought it up

👍 8 [Reply](#)

Evan King **Admin** • 11 months ago

yah probably :)

👍 0 [Reply](#)

[Show All Comments](#)

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

[Schedule A Mock Interview](#)

Questions

[Meta SWE Interview Questions](#)

[Amazon SWE Interview Questions](#)

[Google SWE Interview Questions](#)

[OpenAI SWE Interview Questions](#)

[Engineering Manager \(EM\) Interview Questions](#)

Learn

[Learn System Design](#)

[Learn DSA](#)

[Learn Behavioral](#)

[Learn ML System Design](#)

[Learn Low Level Design](#)

[Guided Practice](#)

Links

[FAQ](#)

[Pricing](#)

[Gift Mock Interviews](#)

[Gift Premium](#)

[Become a Coach](#)

[Our Coaches](#)

[Hello Interview Premium](#)

Legal

[Terms and Conditions](#)

[Privacy Policy](#)

Contact

[About Us](#)

[Product Support](#)

7511 Greenwood Ave North

Unit #4238 Seattle

WA 98103